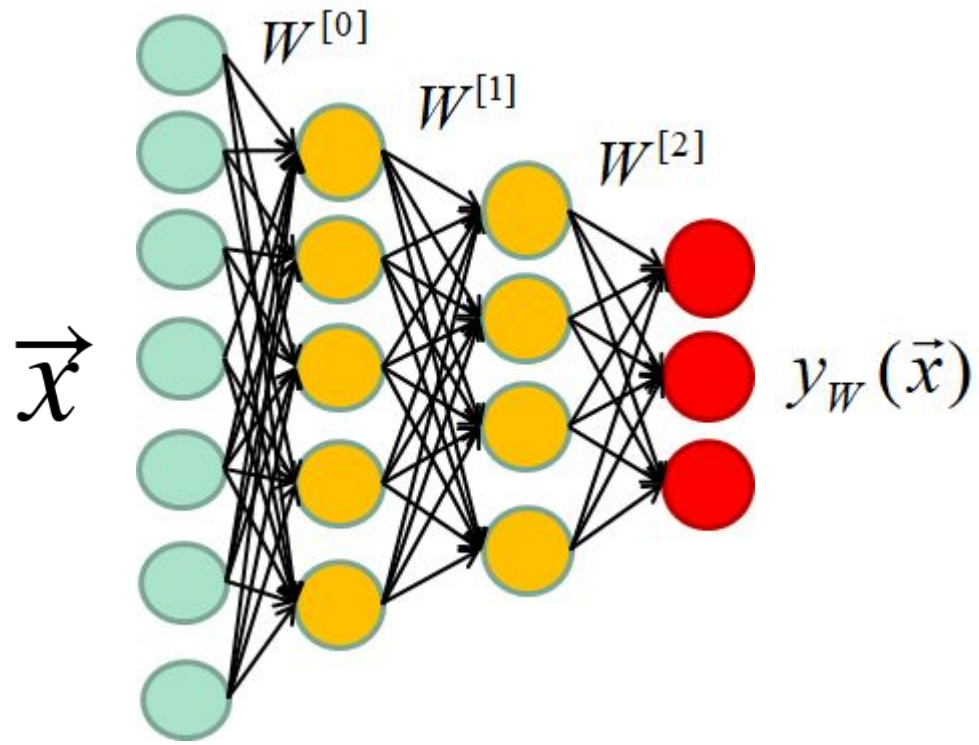


Deep learning for medical imaging school

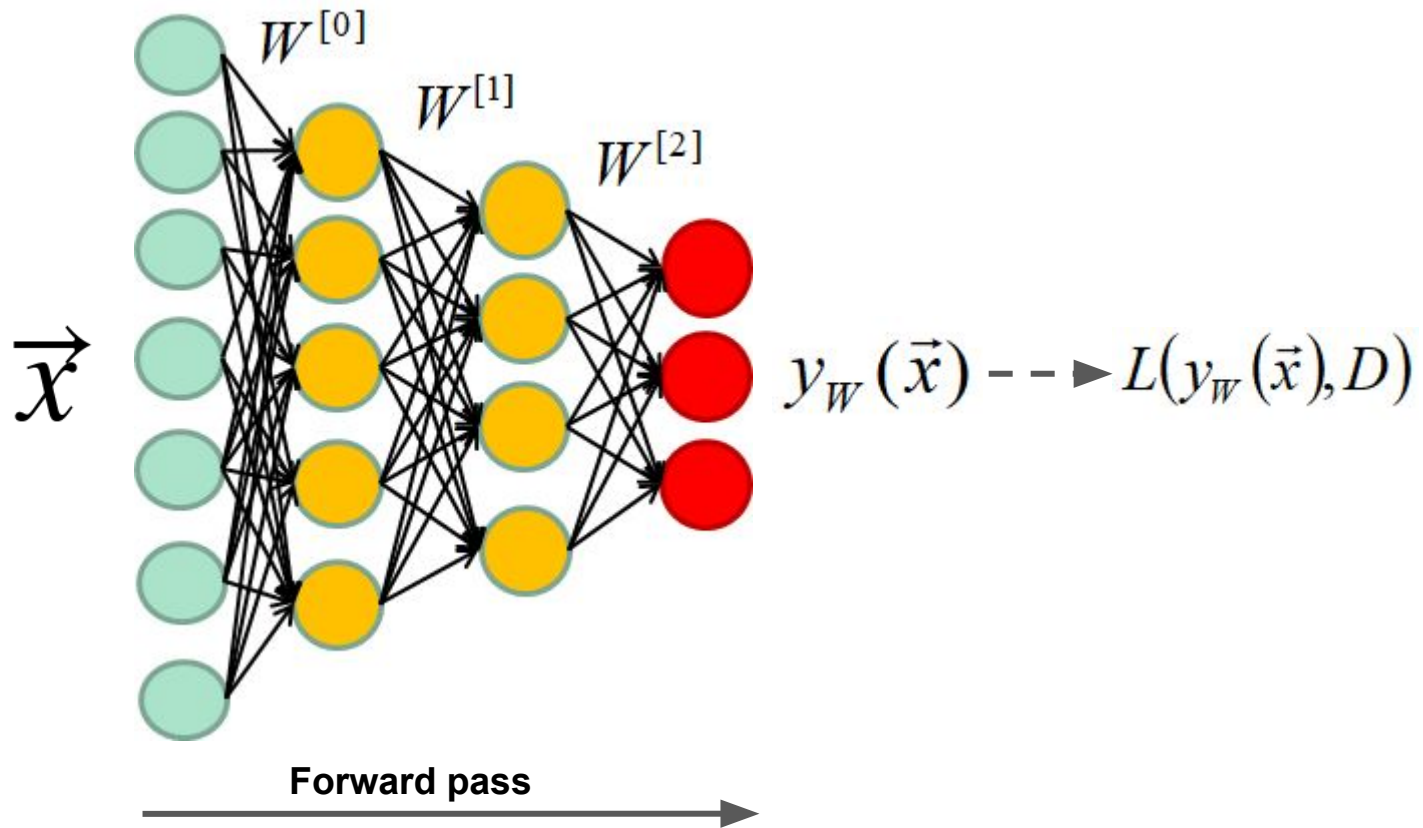
Hands-on session

Autoencoders

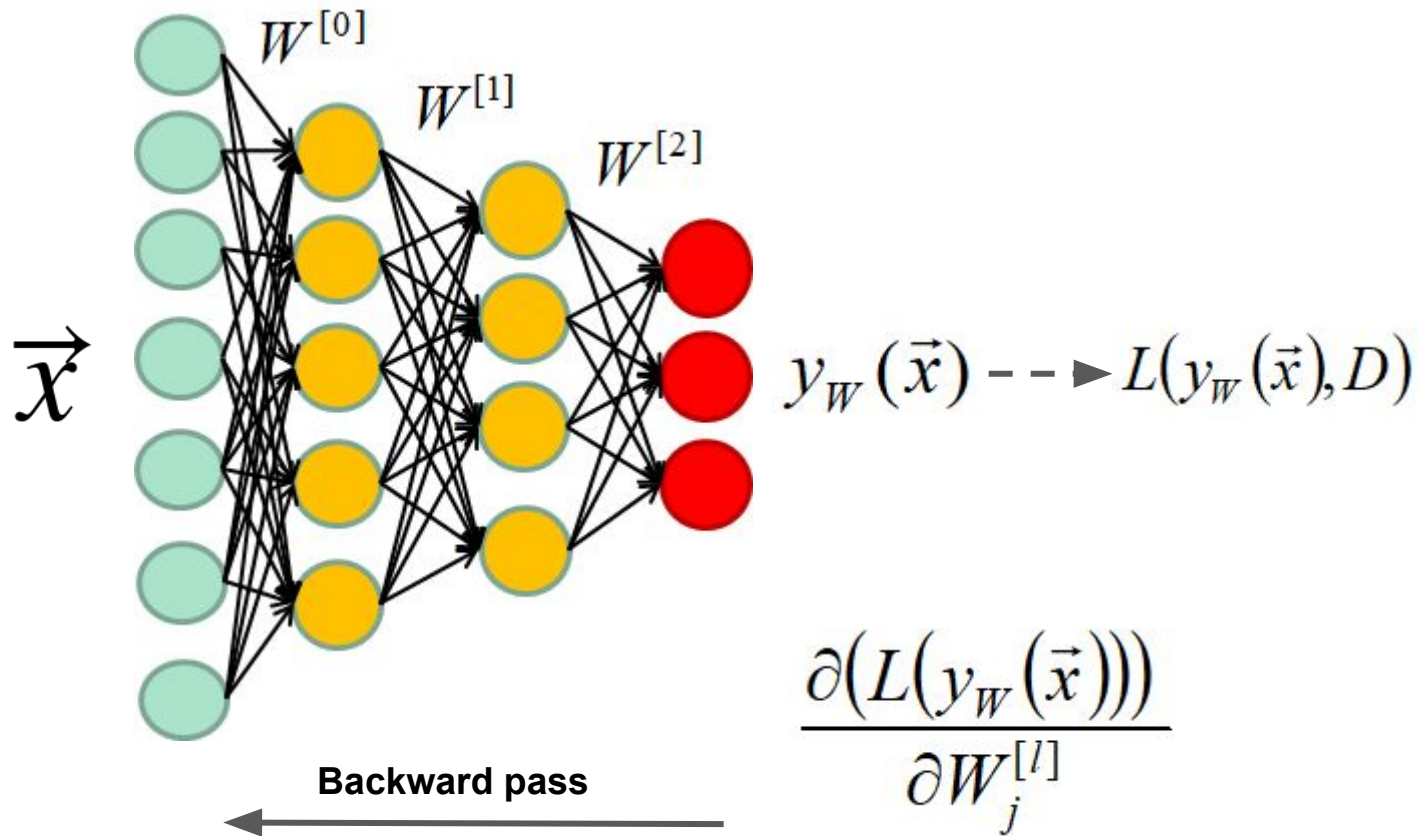
By
Pierre-Marc Jodoin, Thierry Judge



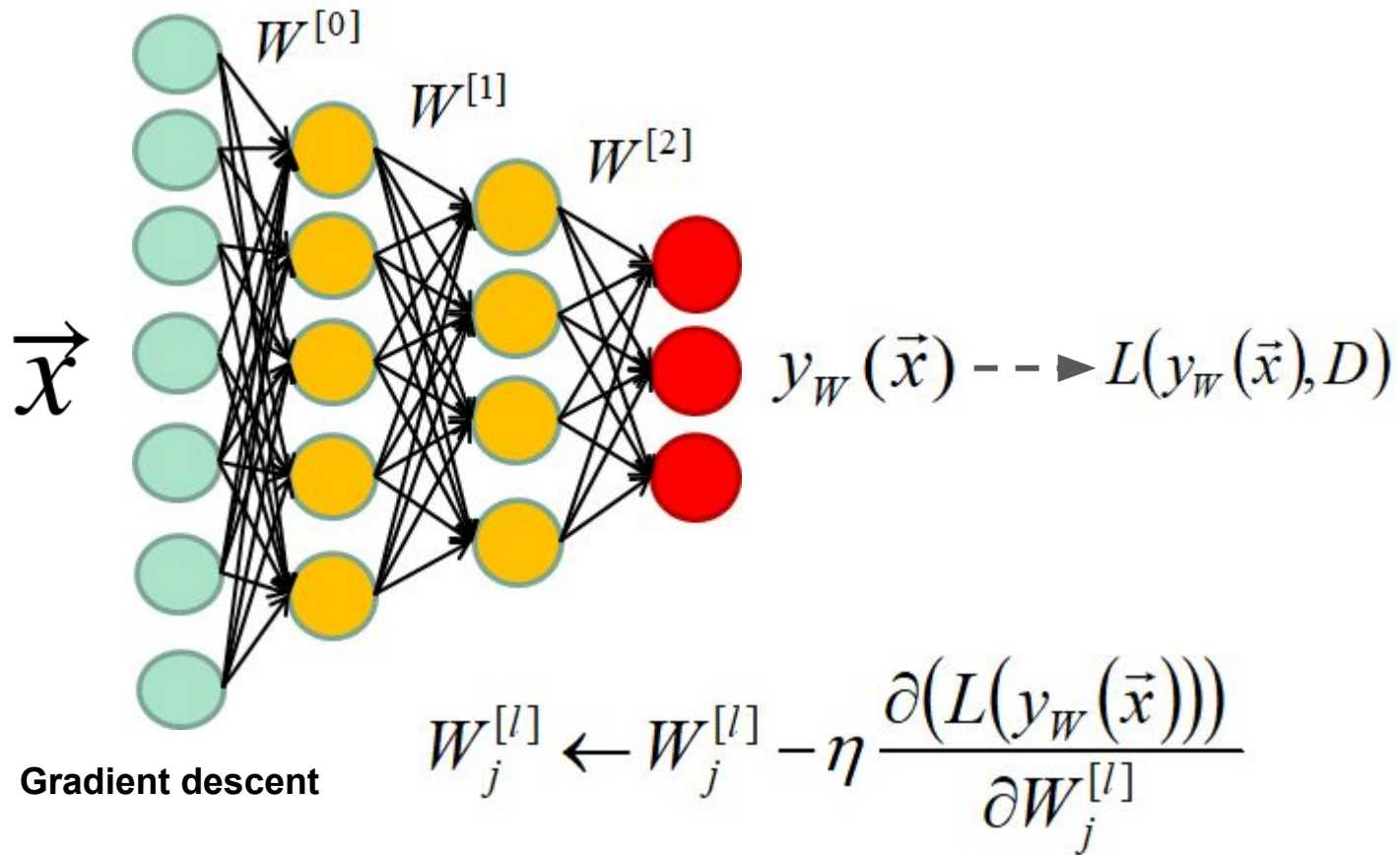
Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}$

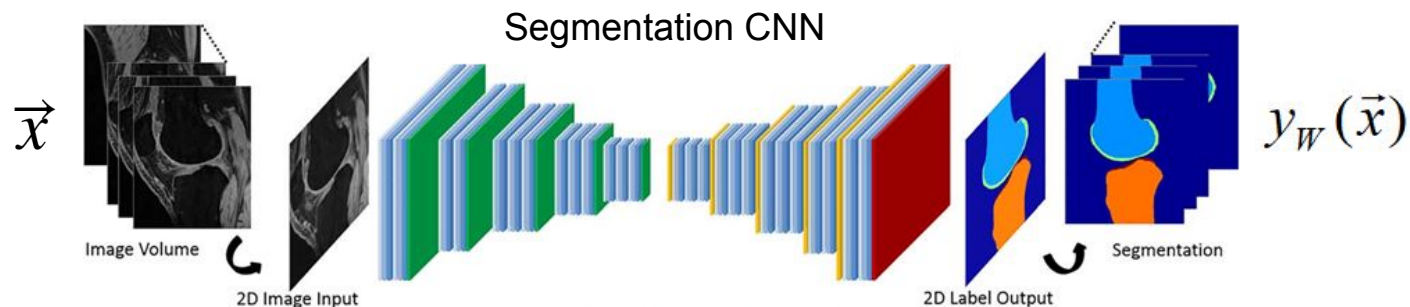
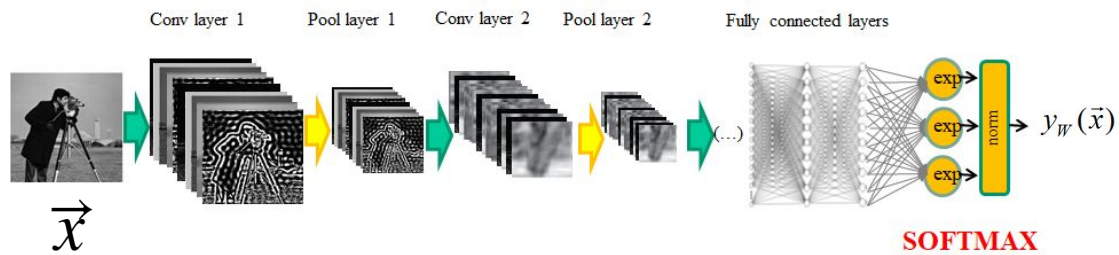
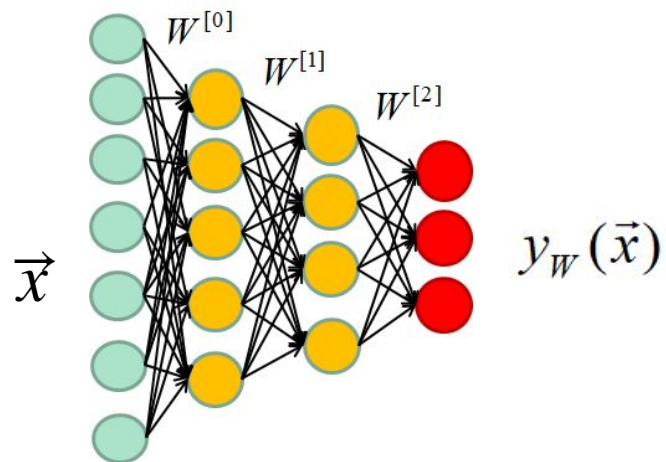


Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}$



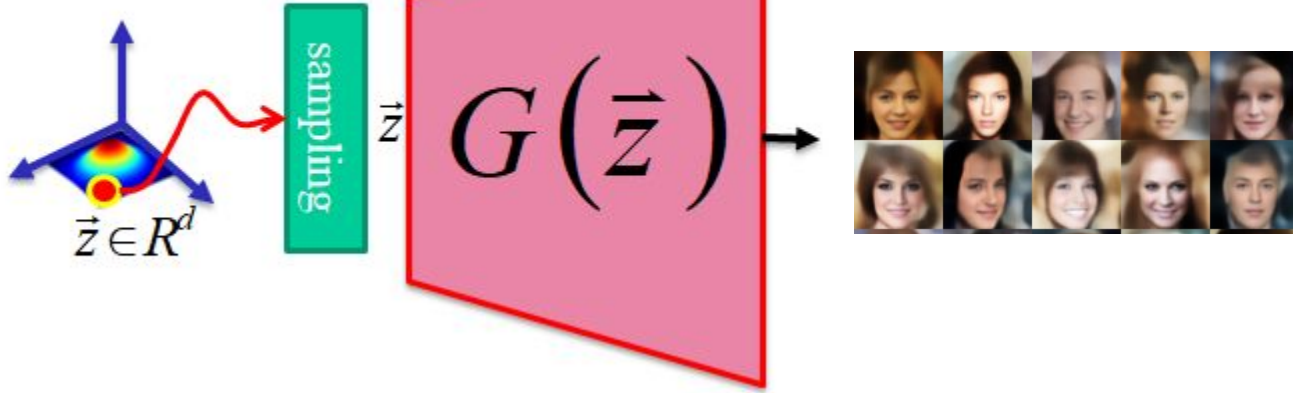
Annotated dataset: $D = \{(\vec{x}_1, t_1), (\vec{x}_2, t_2), \dots, (\vec{x}_N, t_N)\}$





GAN

(Latent space)

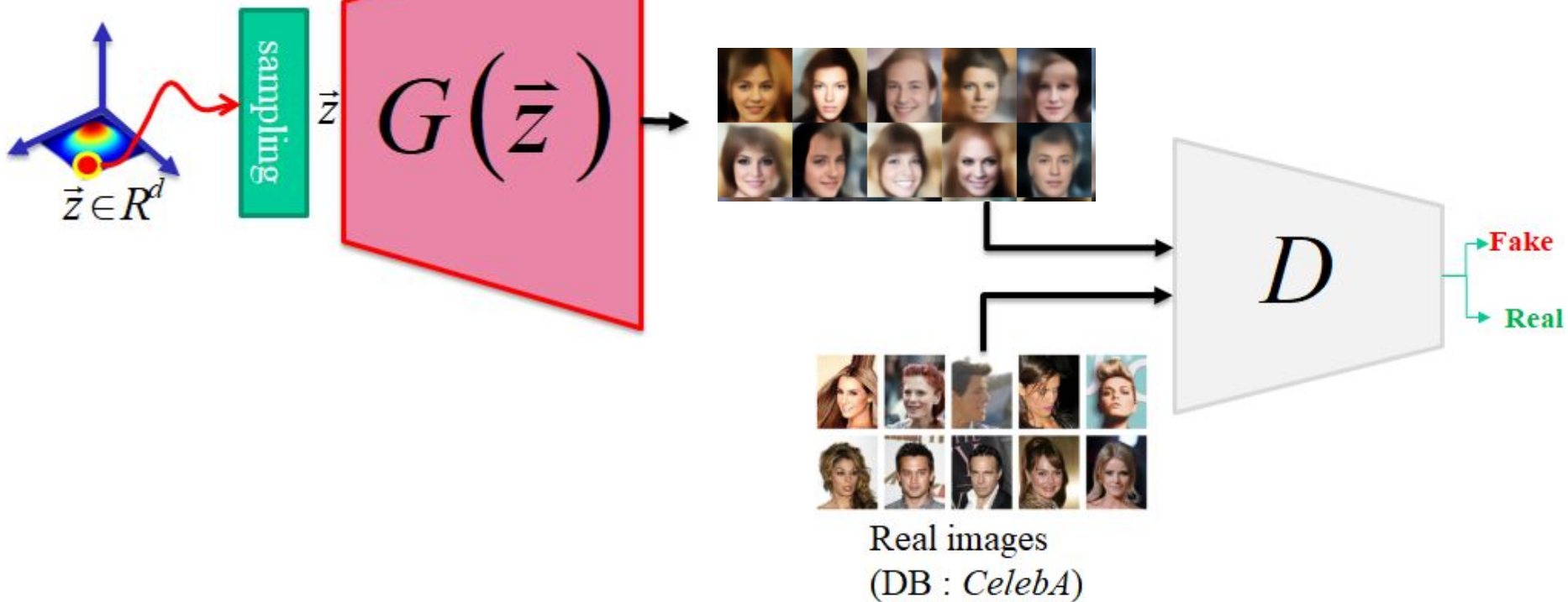


Loss?

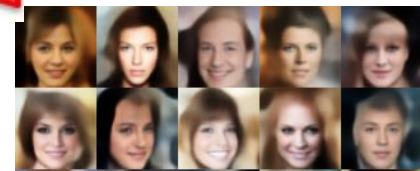
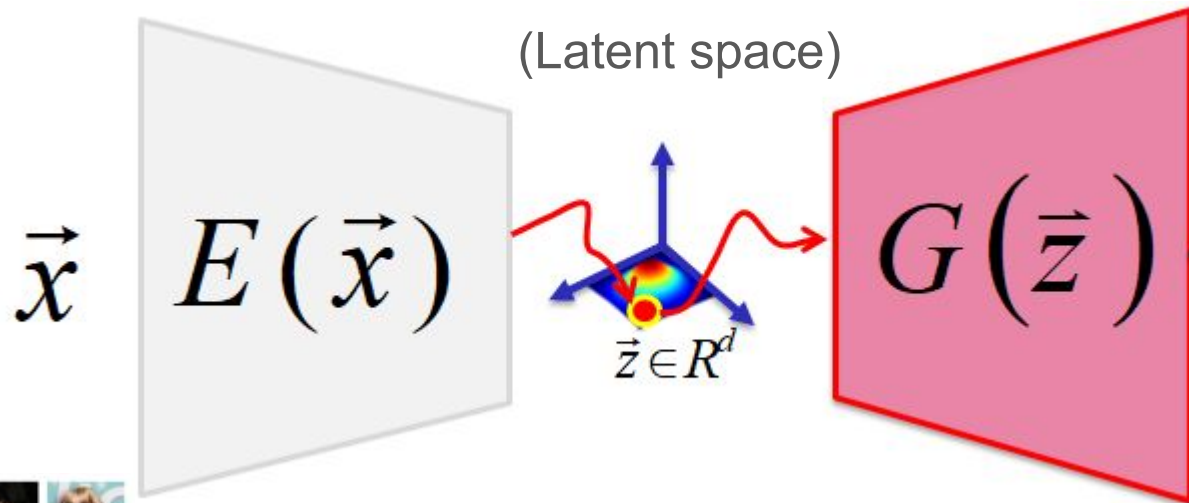
Annotated dataset?

GAN

(Latent space)

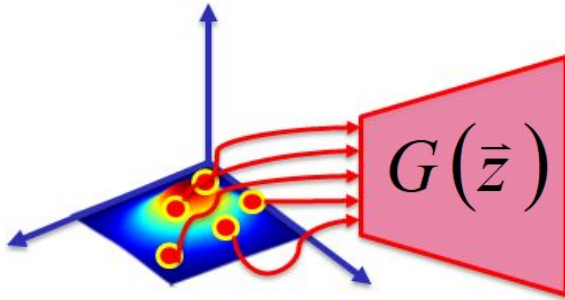


Autoencoders



Autoencoders (once training is over)

(Latent space)



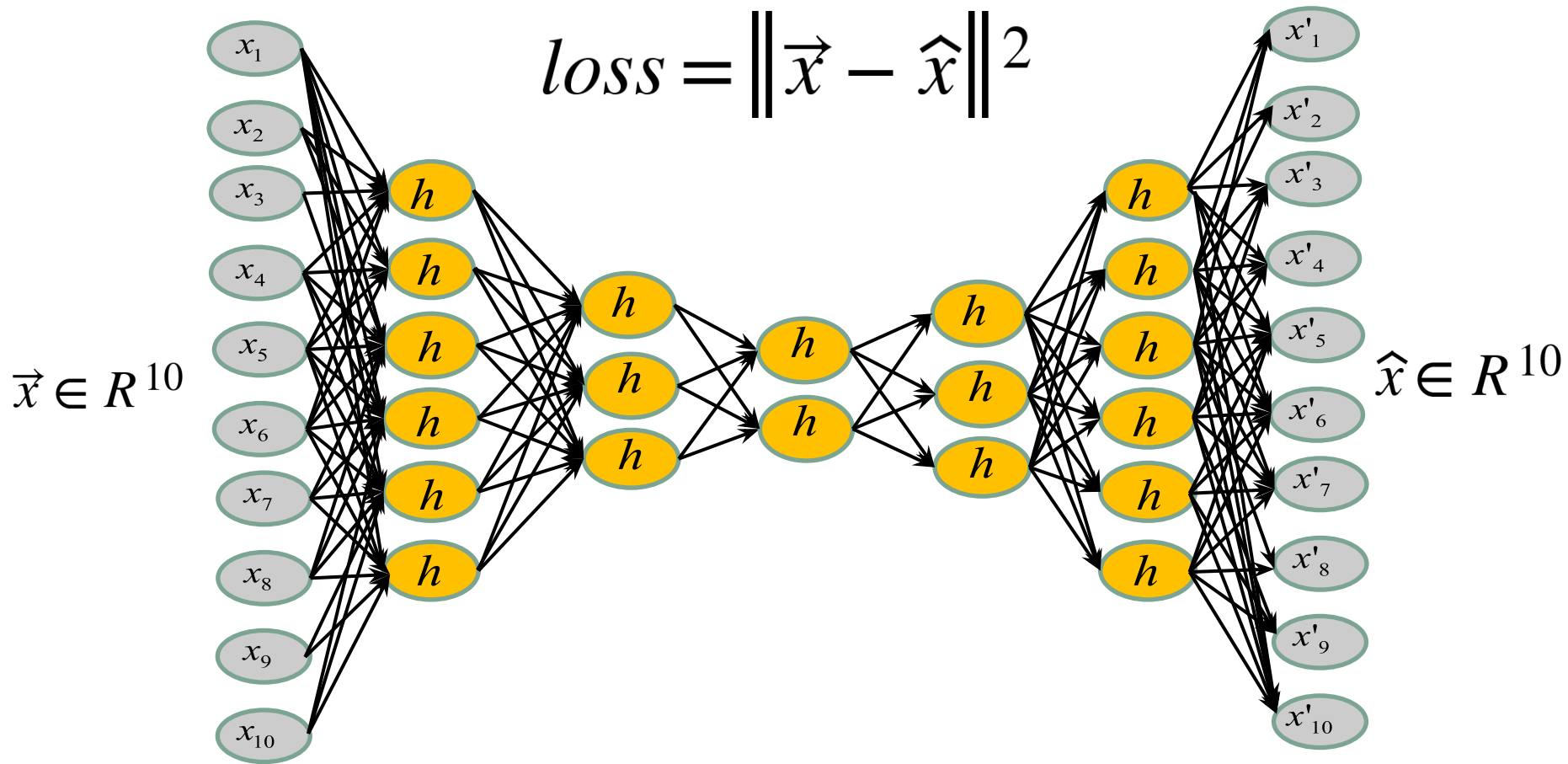
Summary

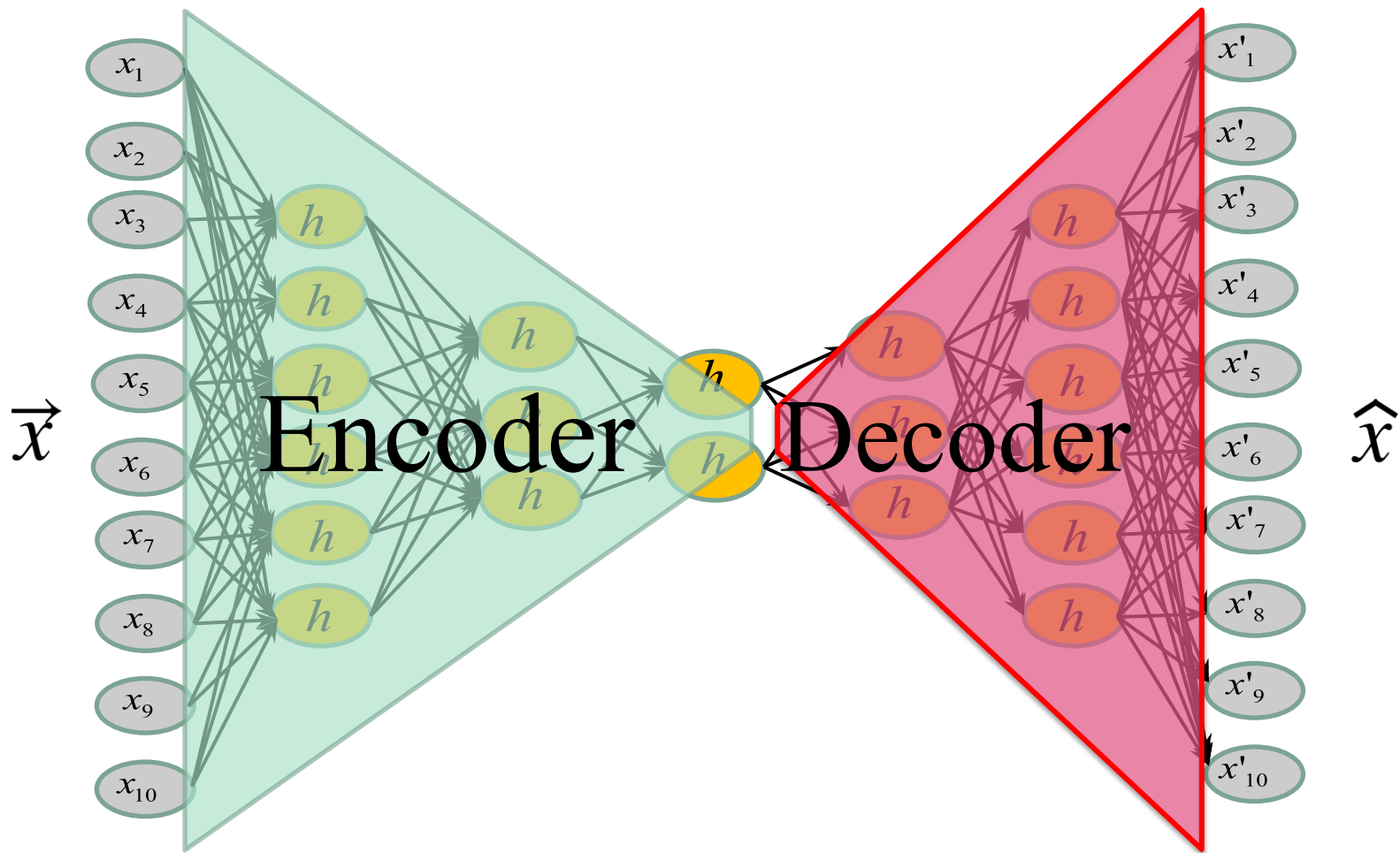
1. What are autoencoders and variational autoencoders?
2. How do they apply to MNIST (grayscale images)?
3. How do they apply to segmentation maps (ACDC cardiac labels)?

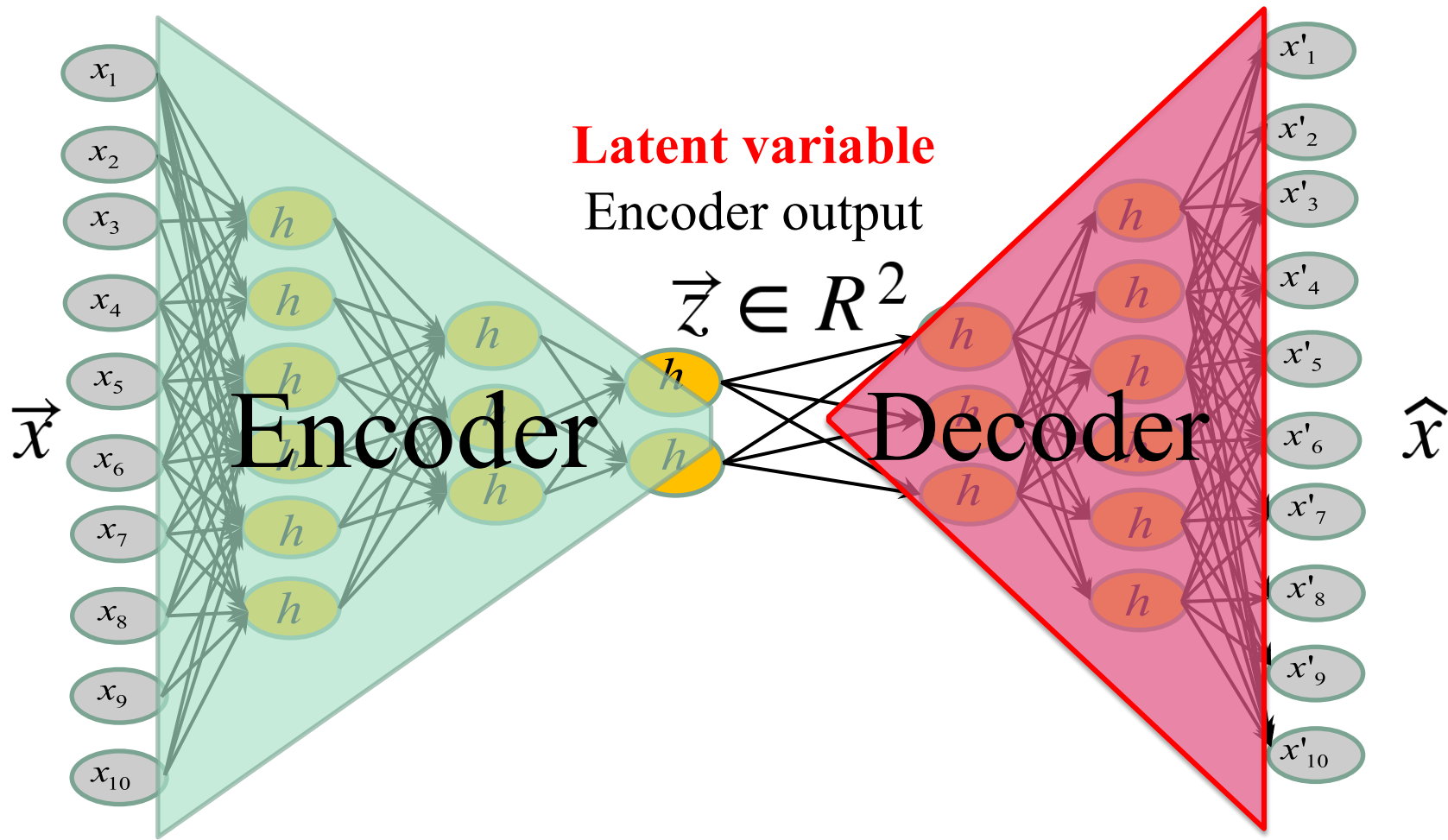
Goal : learn the latent representation of a set of data

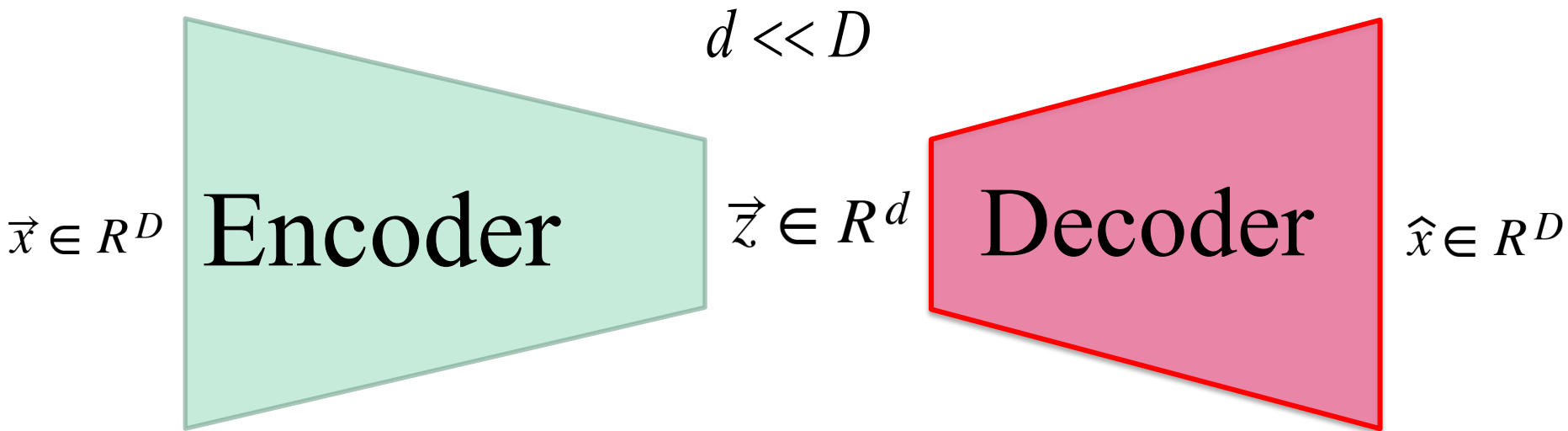
How : by training a Neural Net to output its own ...
input!

Note : if you are familiar with AE and VAE, you may skip the next couple of slides.



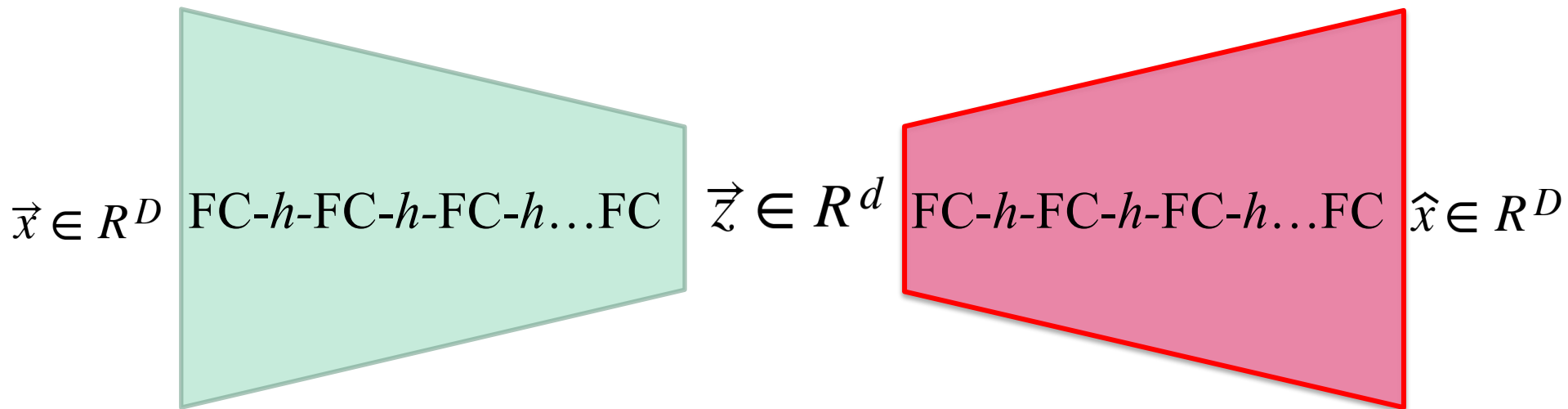






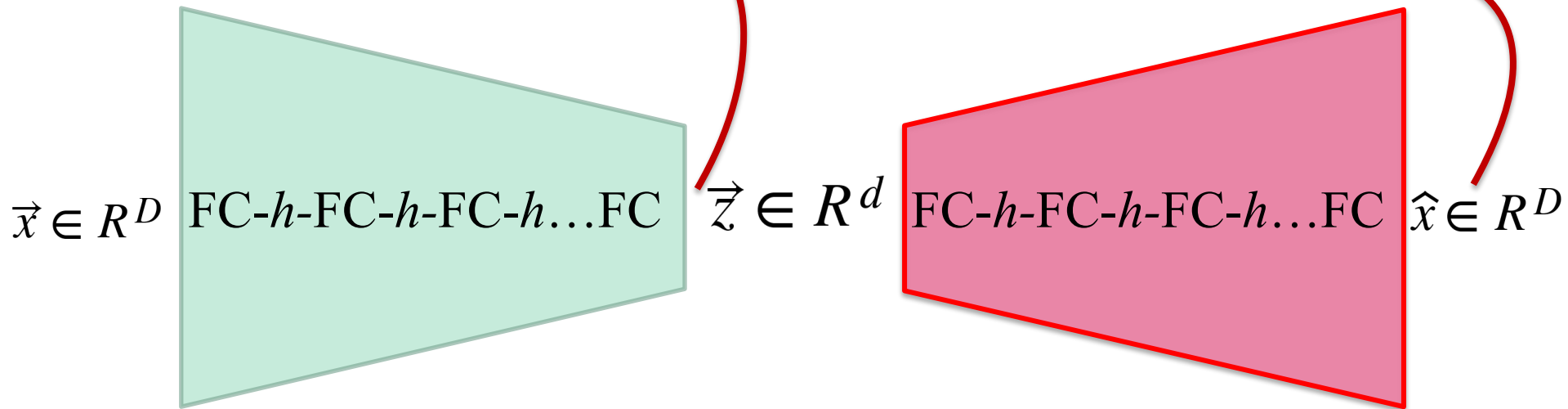
Fully-Connected Layers

h : activation function



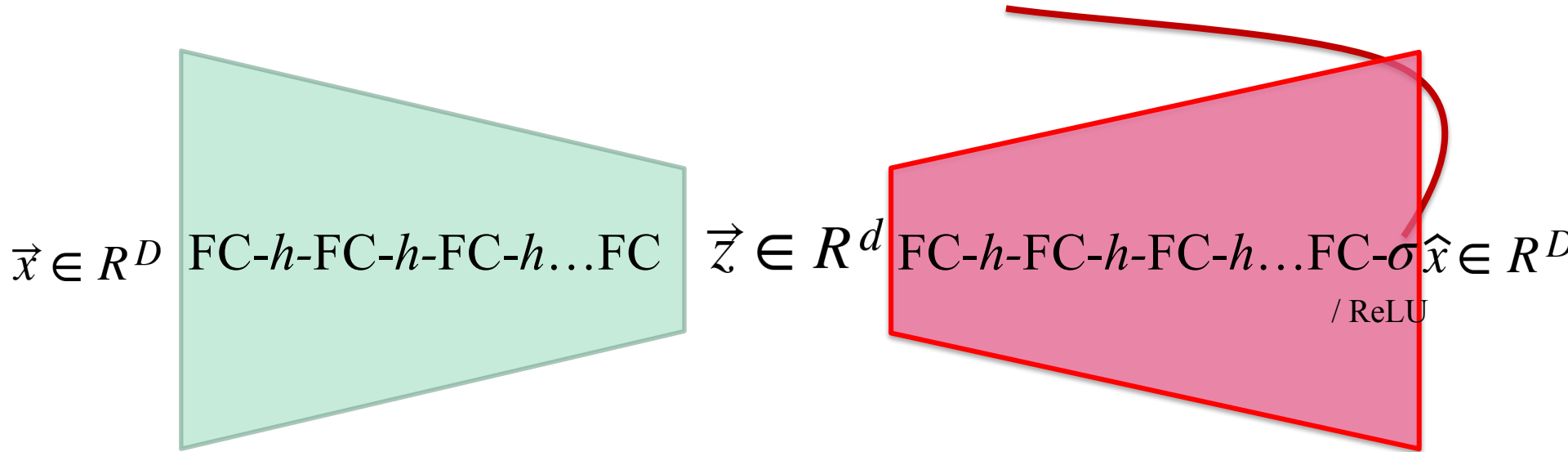
Very often...

No activation function at the output
of the encoder and the decoder



See **why?**

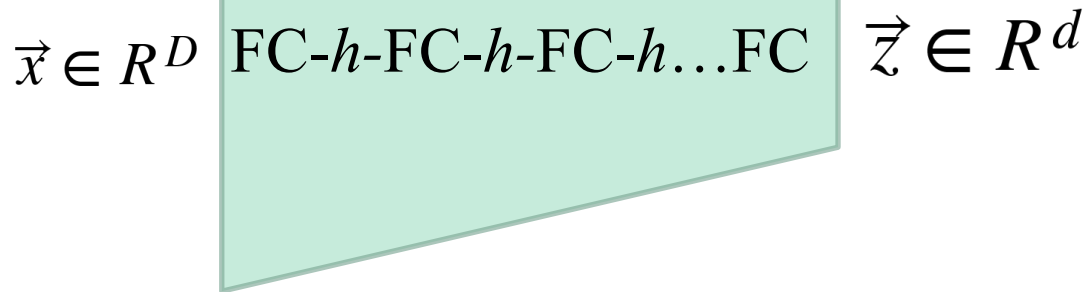
Sometimes **sigmoid** to predict pixel values between 0 and 1 or **ReLU** when the pixel values can be large but never negative.



The number of neurons

Decrease (or stay the same)

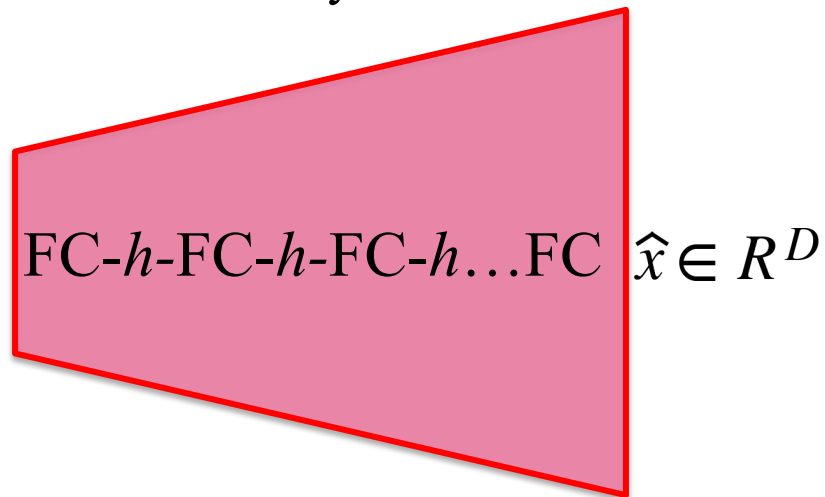
from a layer to another



The number of neurons

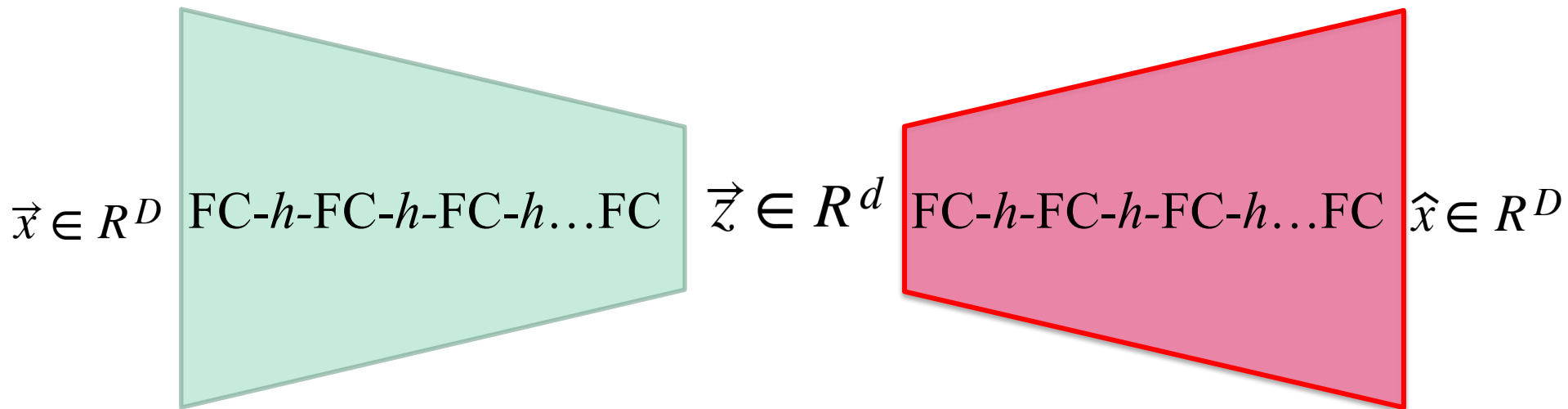
Increase (or stay the same)

from a layer to another

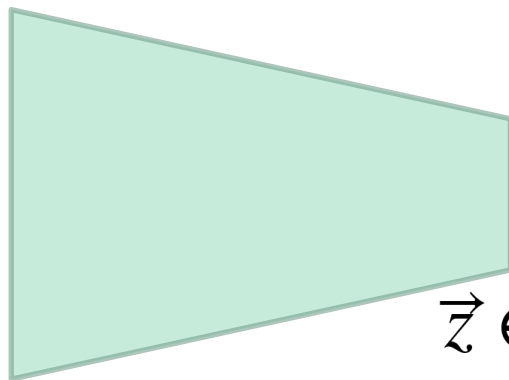
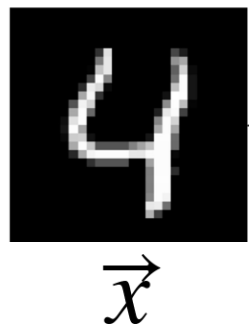


Very often...

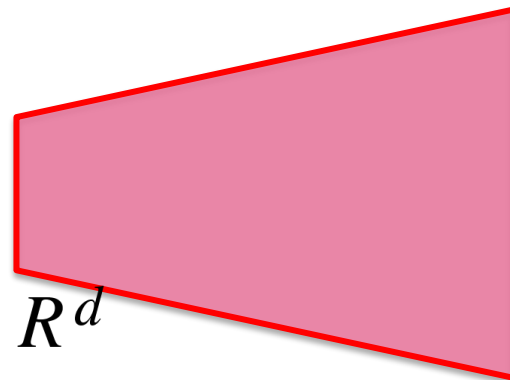
The structure of the encoder is the **dual** of that of the decoder



Basic image-based autoencoder



$\vec{z} \in R^d$



Simple MNIST Autoencoder

```
class autoencoder(nn.Module):  
    def __init__(self):  
        super(autoencoder, self).__init__()  
        self.encoder = nn.Sequential(  
            nn.Linear(28 * 28, 128), nn.ReLU(True),  
            nn.Linear(128, 64), nn.ReLU(True),  
            nn.Linear(64, 12), nn.ReLU(True),  
            nn.Linear(12, 2))  
        self.decoder = nn.Sequential(  
            nn.Linear(2, 12), nn.ReLU(True),  
            nn.Linear(12, 64), nn.ReLU(True),  
            nn.Linear(64, 128), nn.ReLU(True),  
            nn.Linear(128, 28 * 28))  
  
    def forward(self, x):  
        z = self.encoder(x)  
        x_prime = self.decoder(z)  
        return x_prime
```

Latent space 2D



Simple MNIST Autoencoder

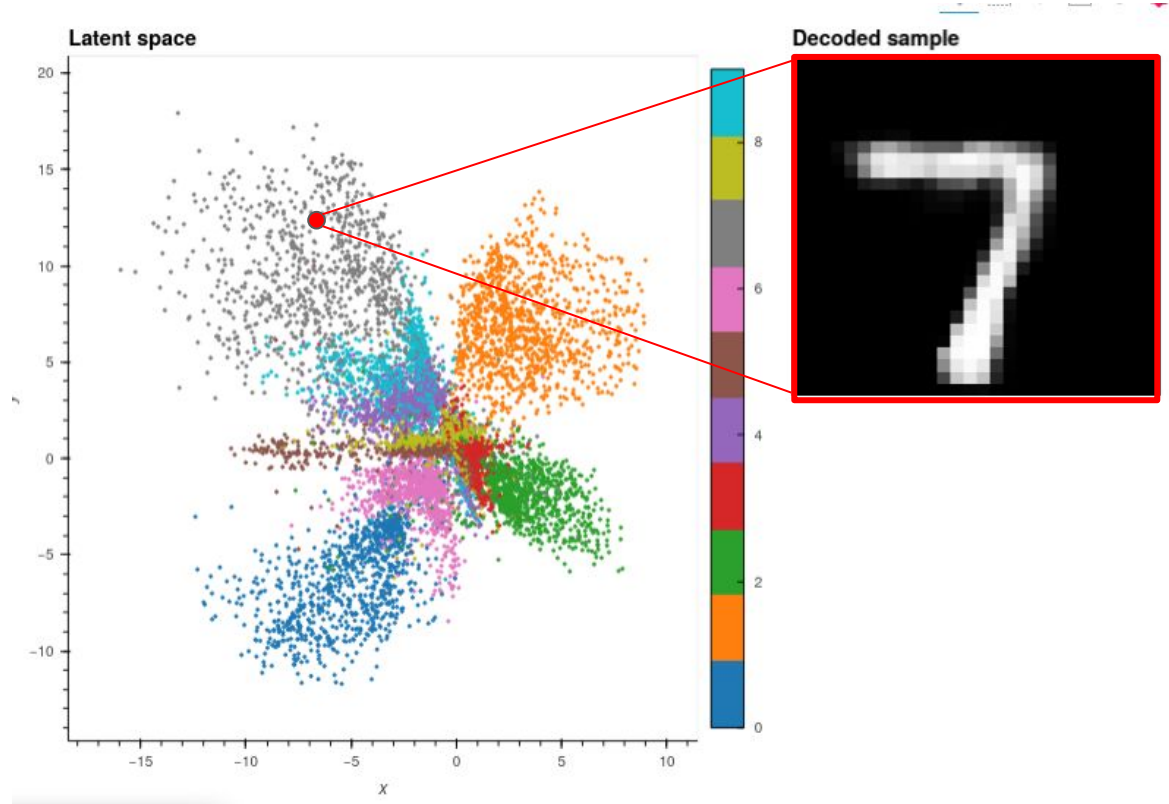
```
class autoencoder(nn.Module):  
    def __init__(self):  
        super(autoencoder, self).__init__()  
        self.encoder = nn.Sequential(  
            nn.Linear(28 * 28, 128), nn.ReLU(True),  
            nn.Linear(128, 64), nn.ReLU(True),  
            nn.Linear(64, 12), nn.ReLU(True),  
            nn.Linear(12, 2))  
        self.decoder = nn.Sequential(  
            nn.Linear(2, 12), nn.ReLU(True),  
            nn.Linear(12, 64), nn.ReLU(True),  
            nn.Linear(64, 128), nn.ReLU(True),  
            nn.Linear(128, 28 * 28))  
  
    def forward(self, x):  
        z = self.encoder(x)  
        x_prime = self.decoder(z)  
        return x_prime
```

symmetry

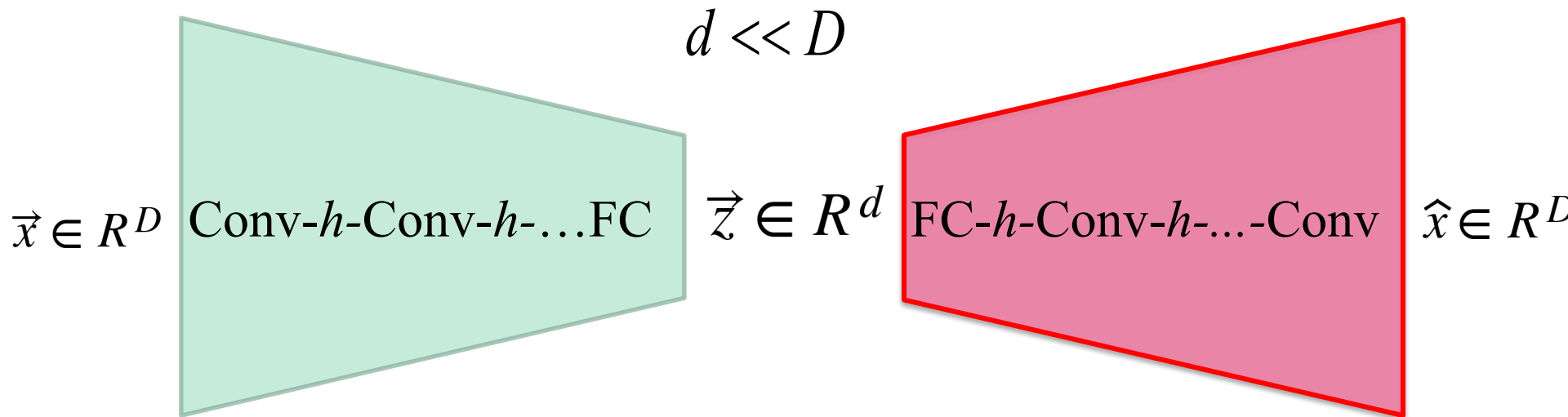


MNIST latent space (for 1000 images)

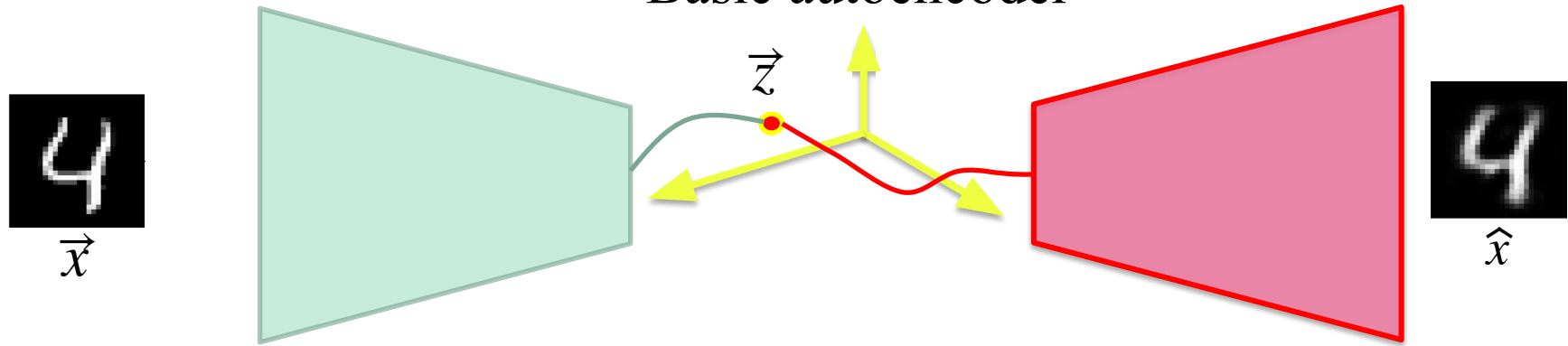
Each 2D point corresponds to an image



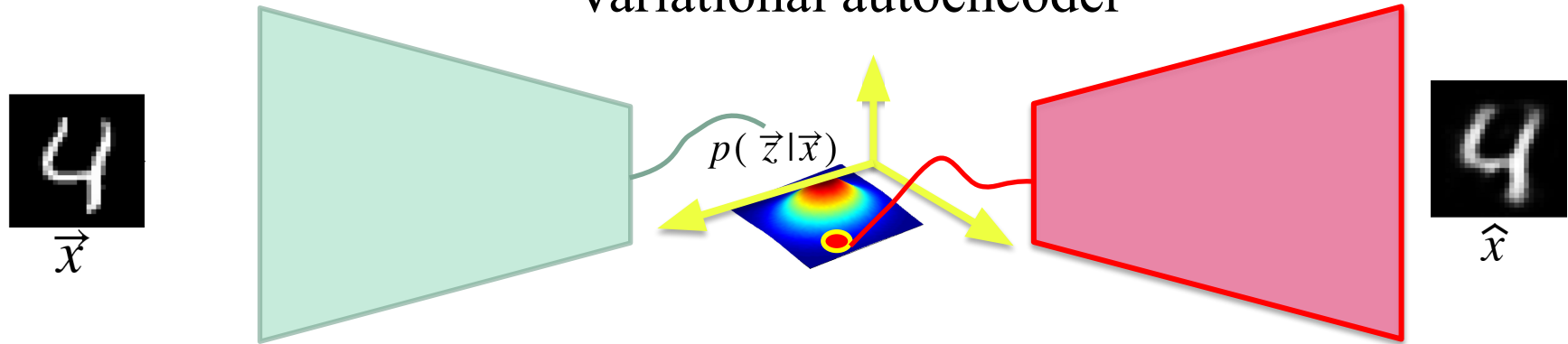
Conv layers



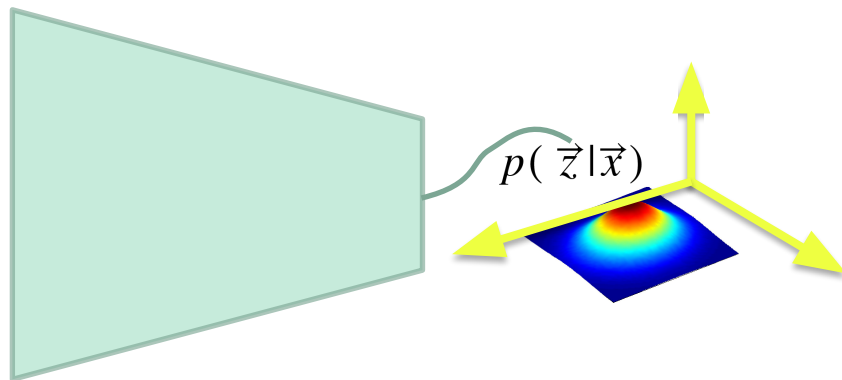
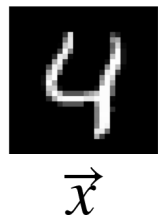
Basic autoencoder



Variational autoencoder

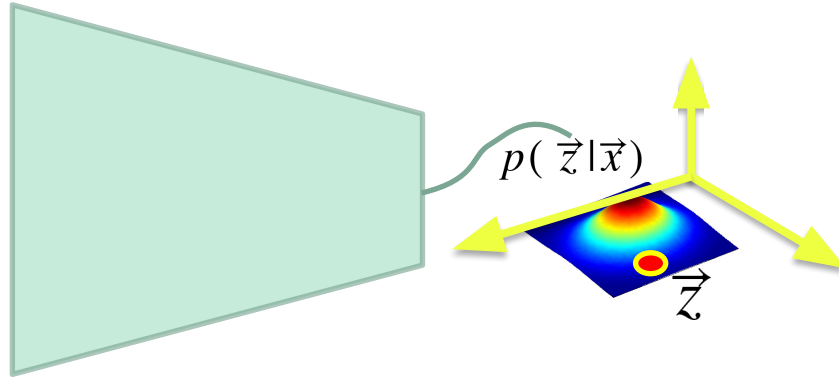
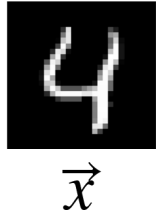


Variational autoencoder



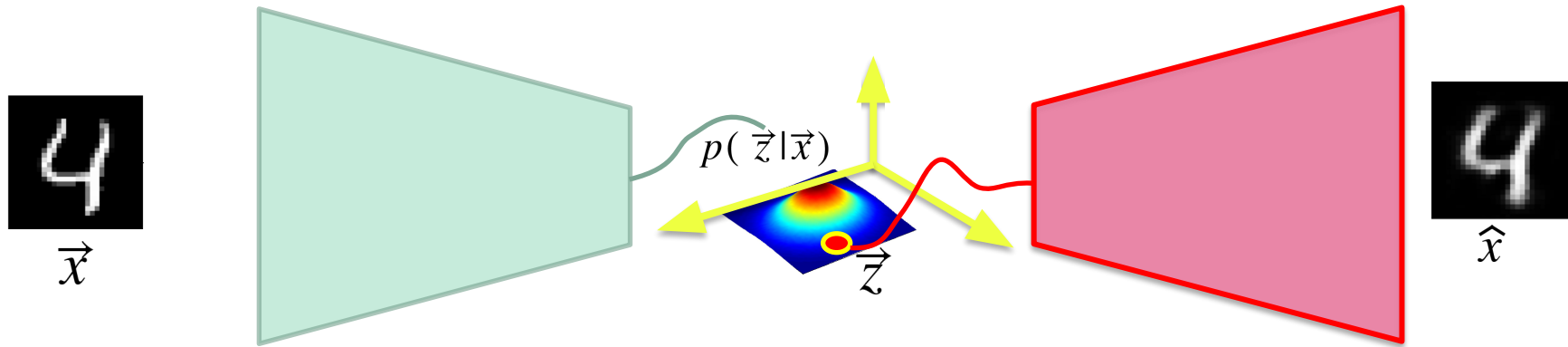
The encoder outputs a **distribution** $p(\vec{z}|\vec{x})$
and not just a **vector** $\vec{z}$

Variational autoencoder



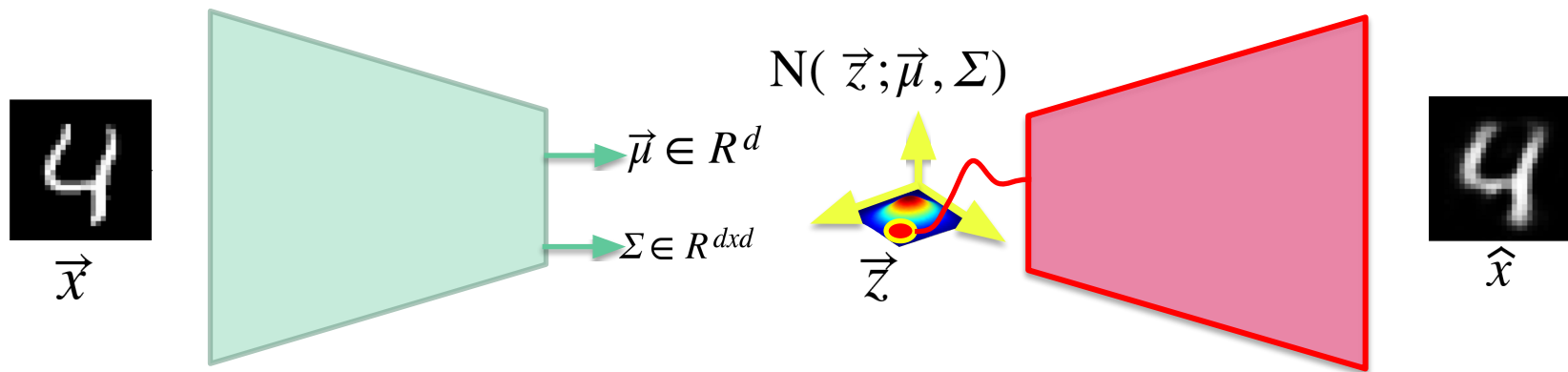
Random sample $\vec{z} \sim P(\vec{z}|\vec{x})$

Variational autoencoder

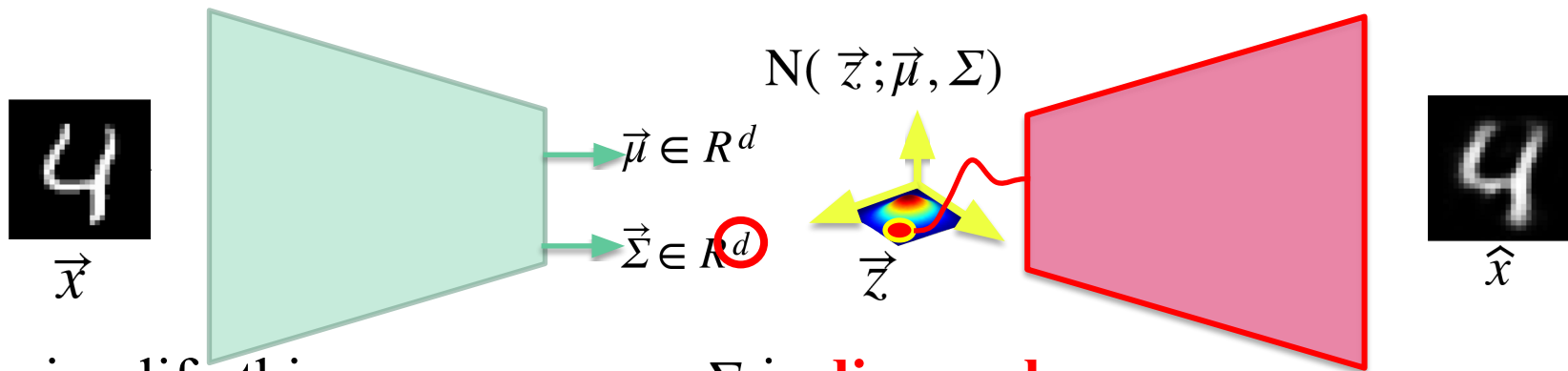


...and rebuild $\hat{x}$

$$p(\vec{z}|\vec{x}) \sim \textit{Gaussian}$$



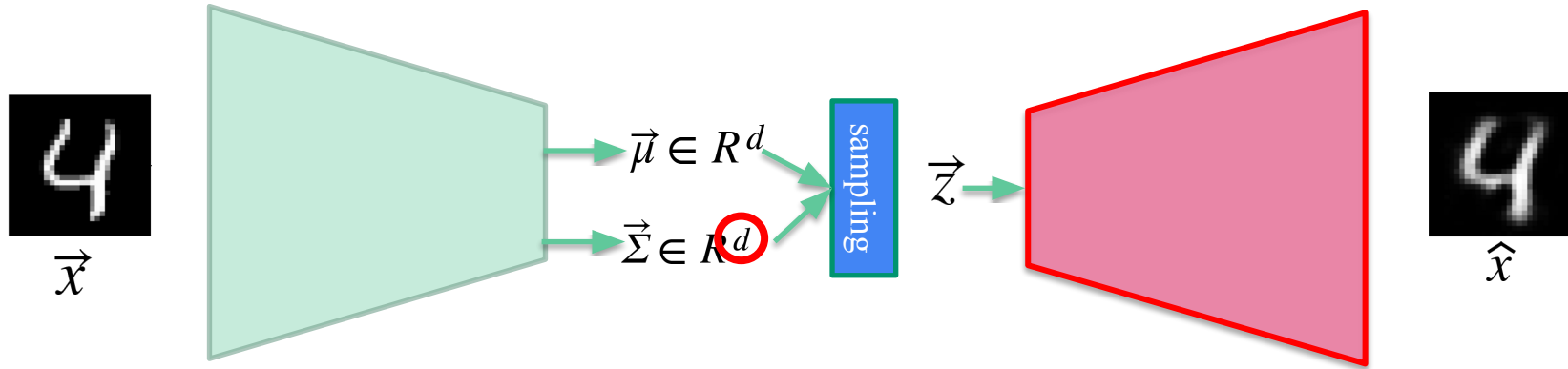
$$p(\vec{z}|\vec{x}) \sim \textit{Gaussian}$$



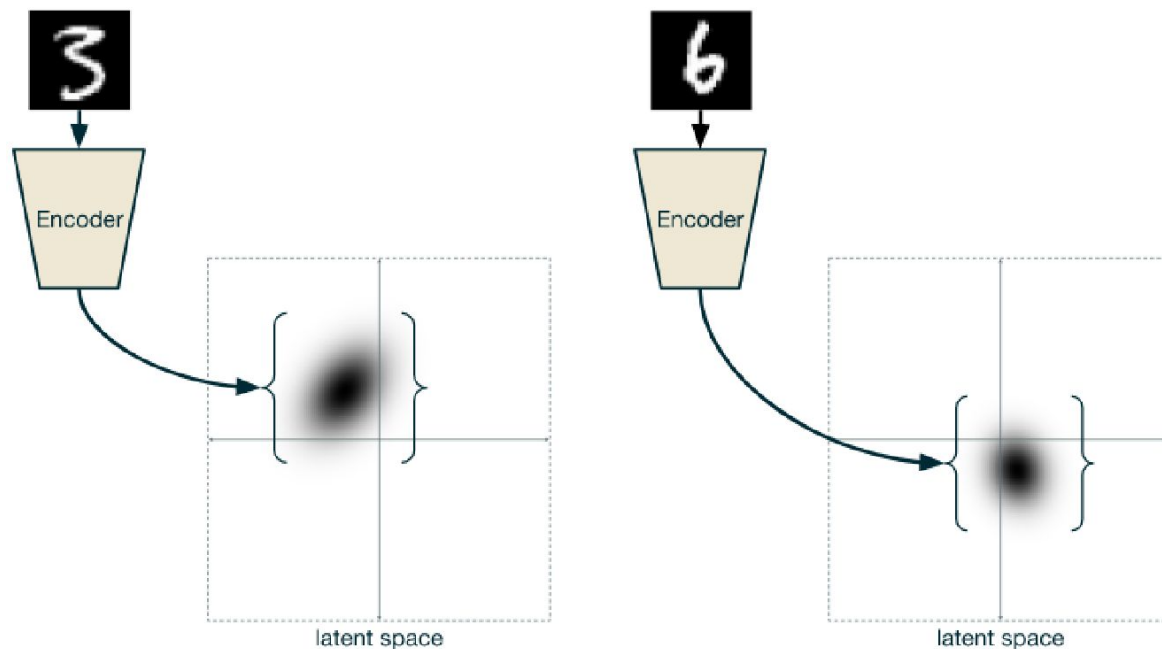
To simplify things, we assume Σ is **diagonal**

$$\Sigma = \begin{pmatrix} \sigma_1^2 & 0 & 0 & 0 \\ 0 & \sigma_1^2 & 0 & 0 \\ 0 & 0 & \ddots & 0 \\ 0 & 0 & 0 & \sigma_d^2 \end{pmatrix}$$

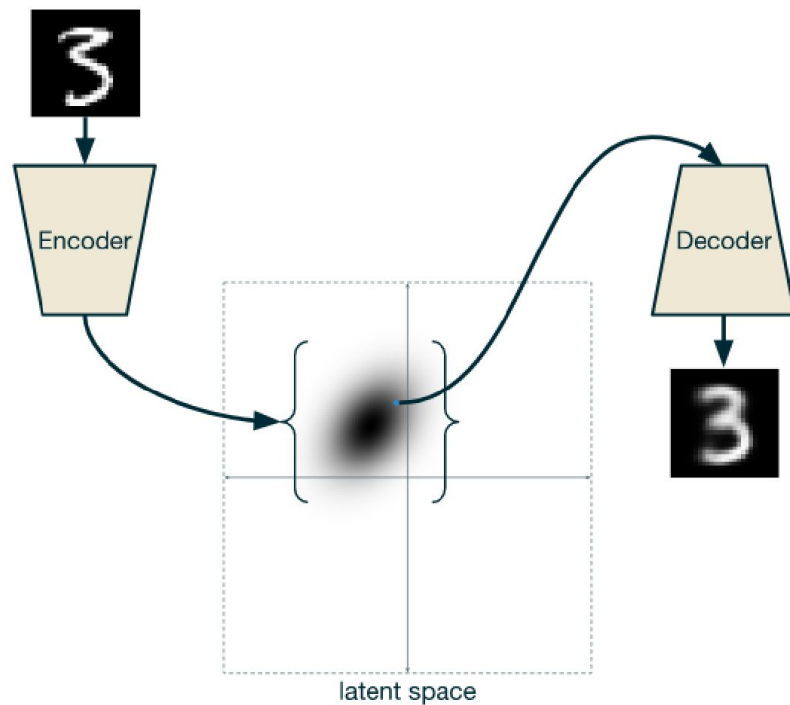
Variational autoencoder



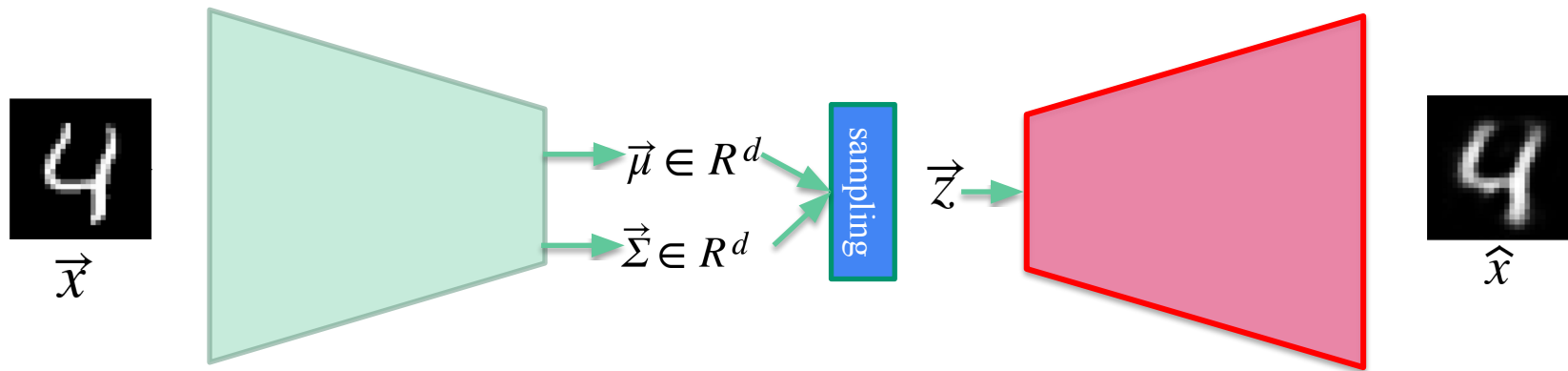
Another way of seeing things...



Another way of seeing things...

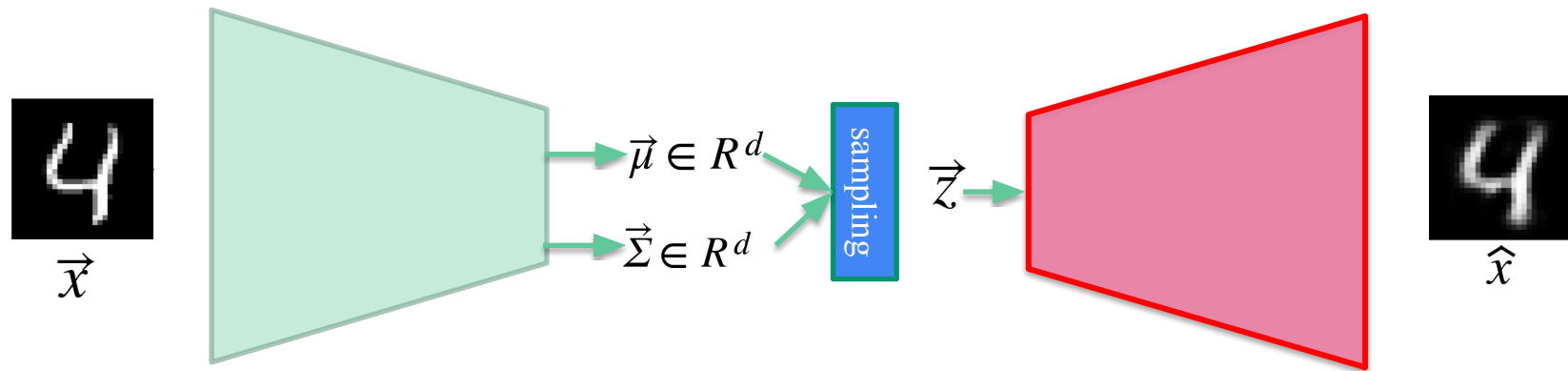


ELBO loss : Evidence Lower Bound loss



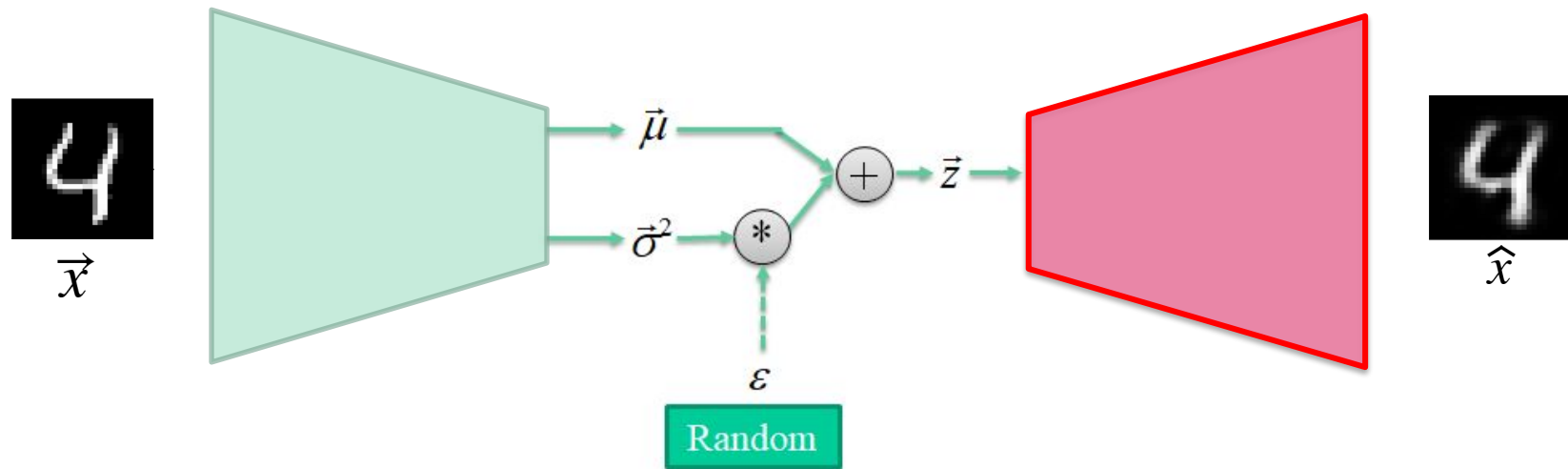
$$Loss = \underbrace{(\vec{x} - \hat{x})^2}_{\text{Loss decoder}} + \lambda \underbrace{KL\left(N(\vec{z}; \vec{0}, \vec{1}), N(\vec{z}; \vec{\mu}, \vec{\Sigma})\right)}_{\text{Loss encoder}}$$

Other loss (in case the output is binary)



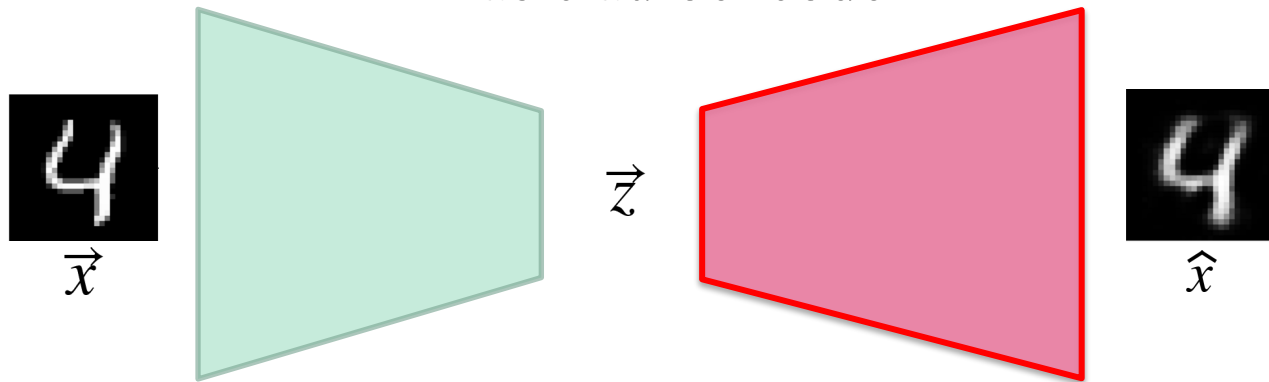
$$\text{Loss} = \underbrace{\text{CrossEntropy}(\vec{x}, \hat{x})}_{\text{Loss decoder}} + \underbrace{\lambda \text{KL}\left(\text{N}(\vec{z}; \vec{0}, \vec{1}), \text{N}(\vec{z}; \vec{\mu}, \vec{\Sigma})\right)}_{\text{Loss encoder}}$$

Reparametrization trick instead of sampling

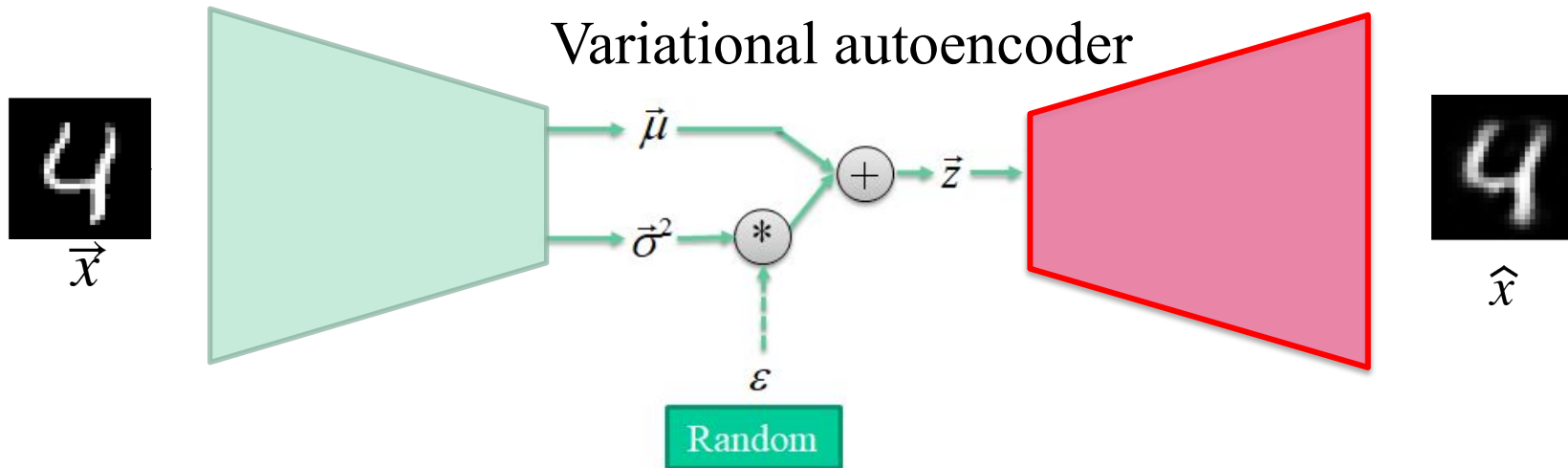


MNIST

Basic autoencoder

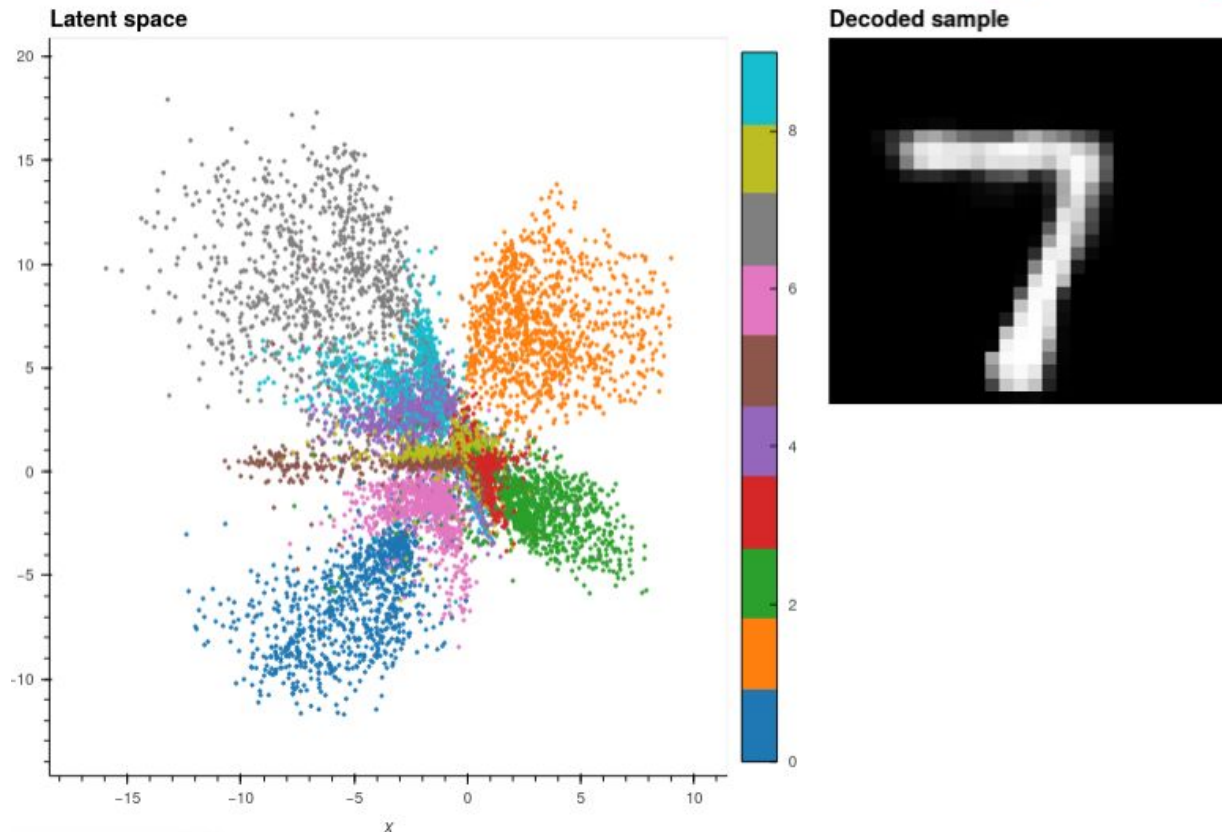


Variational autoencoder



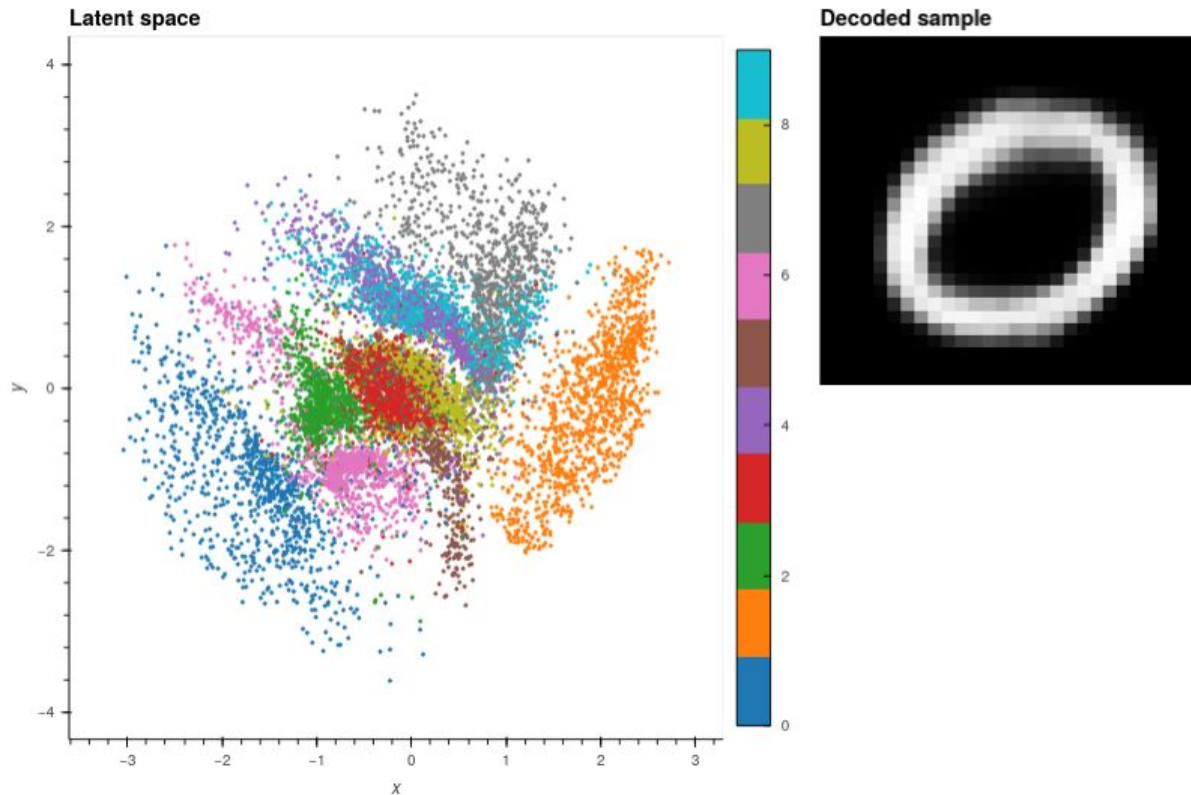
MNIST

Visualize the latent space (autoencoder)



MNIST

Visualize the latent space (variational autoencoder)



MNIST

Code is in the file:

`mnist-autoencoders.ipynb`

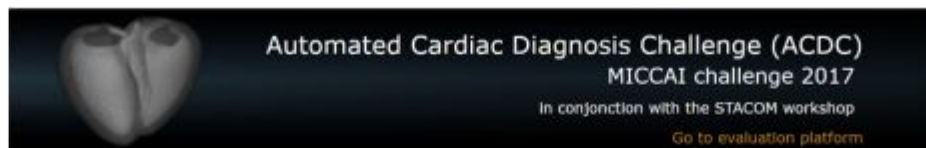
Automated Cardiac Diagnosis Challenge



150 exams (all from different patients) divided into 5 groups:

- dilated cardiomyopathy (DCM),
- hypertrophic cardiomyopathy (HCM),
- myocardial infarction (MINF),
- abnormal right ventricle (RV)
- patients without cardiac disease (NOR).

website: ***creatis.insa-lyon.fr/Challenge/acdc***



Overview
Contest
Participation
Databases
Evaluation
Code
Paper Submission
Contact

Overview

[General context](#) [Scientific interests](#) [Organizers](#)

The goal of this contest is two-fold:

- compare the performance of automatic methods on the segmentation of the left ventricular endocardium and epicardium as the right ventricular endocardium for both end diastolic and end systolic phase instances;
- compare the performance of automatic methods for the classification of the examinations in five classes (normal case, heart failure with infarction, dilated cardiomyopathy, hypertrophic cardiomyopathy, abnormal right ventricle).

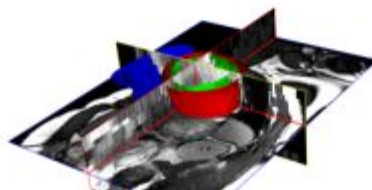
This will be done using a common database of 3D cine-MRI images acquired from 150 patients (30 per pathology plus 30 healthy subjects) and the associated manual references based on the analysis of a clinical expert.

NEWS

4th of April 2017
The full training dataset (100 patients) is available

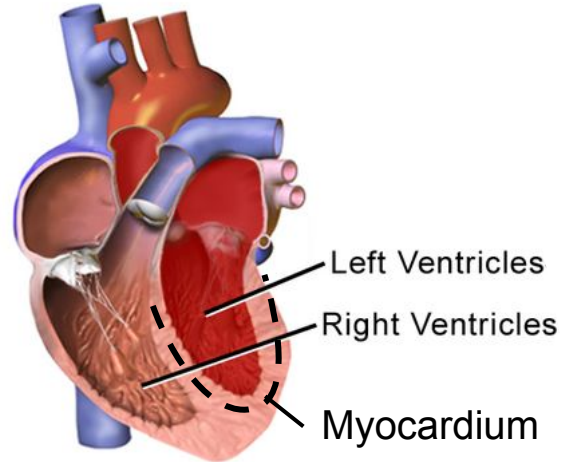
13th of March 2017
Challenge accepted by the MICCAI Satellite Committee

10th of January 2017
The online evaluation platform is operational



Cardiac anatomy

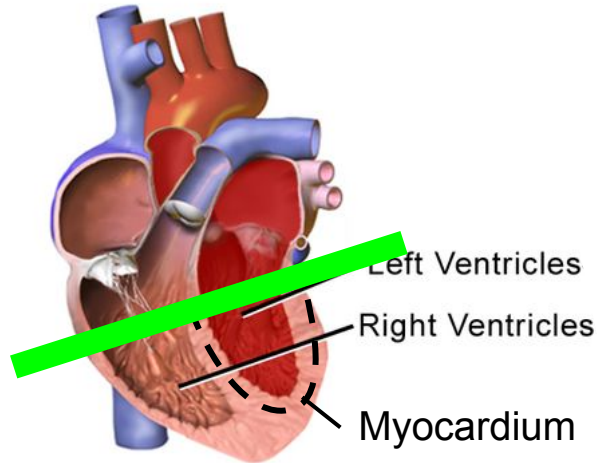
Normal Heart



Chambers relax and fill,
then contract and pump.

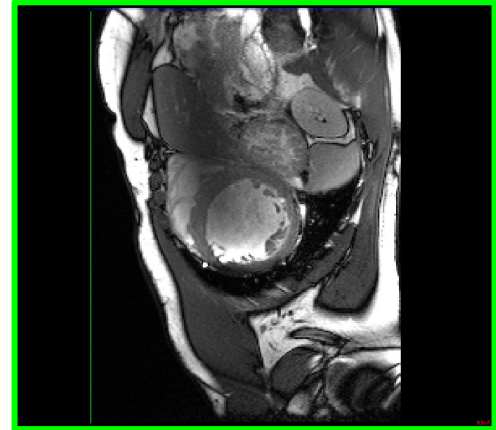
Cardiac anatomy

Normal Heart



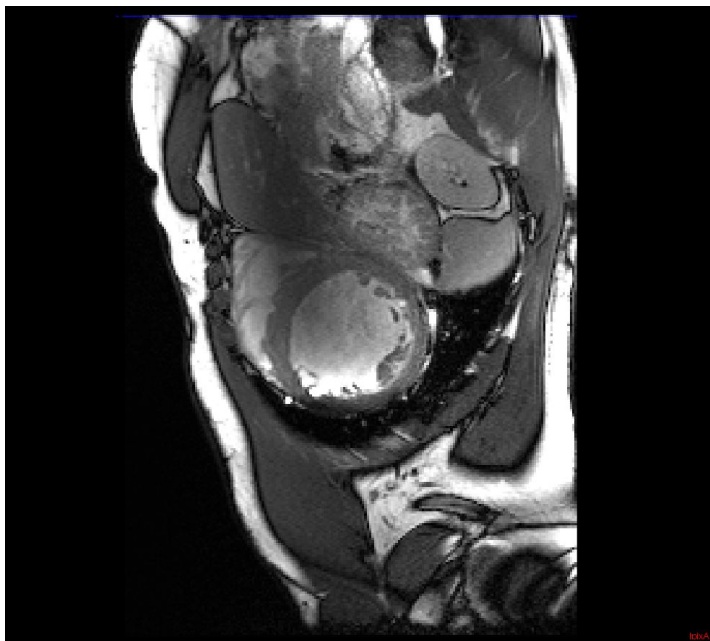
Chambers relax and fill,
then contract and pump.

Cross-section : **short axis view**

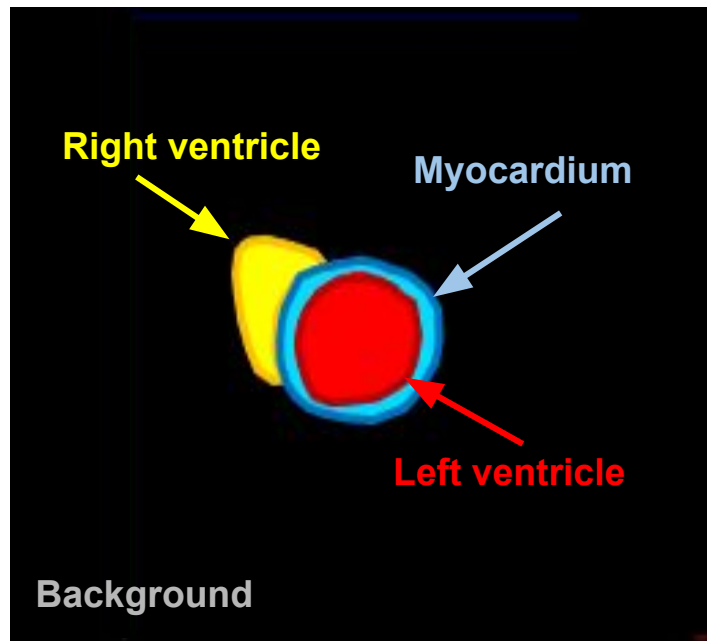




Short axis cardiac MRI

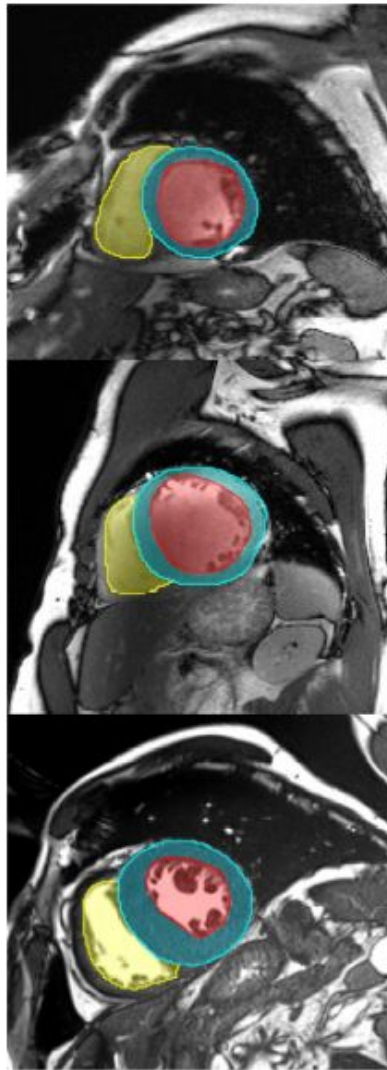


Short axis segmentation map

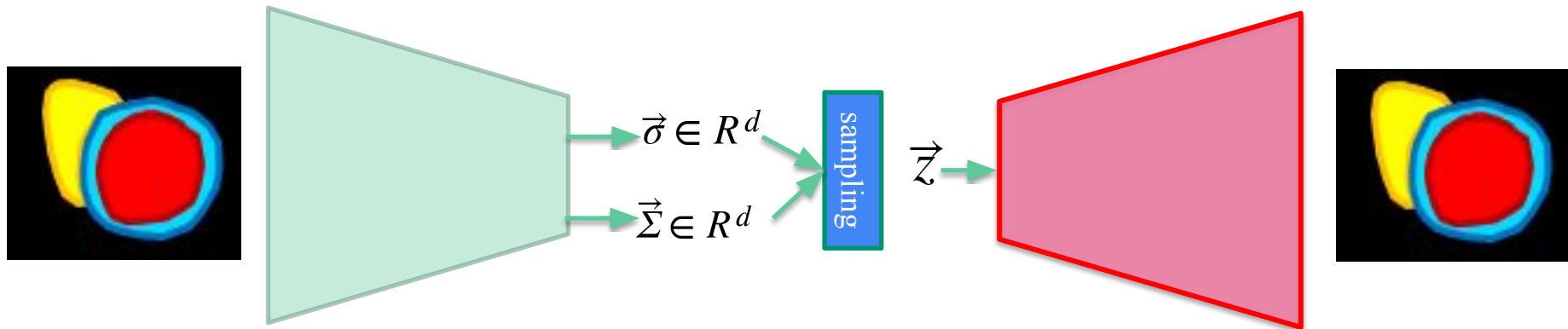




Other examples:



Convolutional Variational autoencoder



ACDC

Code is in the file:

cardiac-mri-autoencoders.ipynb

Thank you